

Portail locataire — rapport FD_CHALLENGE

Dépôt : README.md pour lancer, docs/SYNC.md pour la synchro. Temps passé : **environ 4 h 30**, dont une phase de fondation en série puis trois agents Claude Code en parallèle (un worktree git par lot). Stack : Next.js (App Router) + TypeScript, SQLite via Drizzle, vitest.

1. Ce qui est construit, et ce que ça change pour le locataire

Le locataire (/) voit en un écran, sans cliquer : qui il est, son logement (adresse, pièces, surface, étage), son bail (référence, rôle, statut, dates), **ce qu'il doit** (solde CHF = Σ débits – Σ crédits, dernières écritures avec statut) et **ce qui arrive** (prochaine échéance, maintenance planifiée sur son immeuble). /bail et /bail/<ref> détaillent loyer, charges, parties et objets loués. Un co-titulaire voit le même bail que le titulaire : « mon bail » est résolu par lease_parties, jamais par le seul primary_rental_unit_id.

L'action réelle : le locataire ouvre une demande d'intervention (/tickets/new : catégorie, titre, description, validation en français), la suit (/tickets, /tickets/<id> avec historique) et y commente. Tout cela est écrit dans notre SQLite (tickets, ticket_comments), rattaché par tenant_ref / lease_ref / unit_ref — des références, pas des UUID, pour survivre à un réimport complet.

Le côté gérance (/admin, trois écrans, pas un back-office) : *Connexions* (qui s'est connecté, quand, succès/échec), *Demandes* (boîte d'arrivée, passage ouverte → en cours → clôturée, le changement de statut apparaît dans la timeline du locataire), *Synchro* (dernières exécutions, curseur, nombre de lignes par table, bouton « Relancer la synchro »).

Pour le locataire, le gain est simple : une réponse en dix secondes à « où j'en suis ? » et un canal pour demander quelque chose qui laisse une trace, au lieu d'un e-mail perdu.

2. Lancement et comptes de démonstration

```
npm install && npm run setup && npm run dev      # http://localhost:5173
npm test                                         # 18 suites, 137 tests, chacun sur sa propre base
```

setup, dev et test fonctionnent depuis un clone nu **sans** .env.local : un instantané ERP de quatre locataires (fixtures/) alimente la base. Seule la synchro en direct exige ERP_API et ERP_PUBLISHABLE_KEY (voir .env.example). Aucune clé n'est dans le dépôt.

Mot de passe commun : portail2026

Compte	Rôle	Ce qu'il montre
lea.martin@example.ch	locataire TEN-00005	le tableau de bord complet (BAIL-000005, Ecublens)
lucas.martin@example.ch	co-titulaire du même bail	la résolution par lease_parties
adrien.clerc@example.ch	locataire TEN-00170	l'autre côté du test d'isolation
gerance@example.ch	gérance	les trois écrans de pilotage

3. Choix de synchro et d'isolation

Isolation — par construction, puis testée. Toute lecture locataire passe par un seul module, src/db/tenant-queries.ts, dont chaque fonction prend tenantRef **depuis la session** (cookie signé), jamais d'une URL, d'un formulaire ou d'un en-tête. Une référence dans l'URL (/bail/BAIL-000005) ne fait que *restreindre* à l'intérieur des baux du locataire : une référence étrangère donne null → 404. Le test src/auth/isolation.test.ts ouvre deux vraies sessions et fait demander à chacune le bail de l'autre, sur les vrais modules de route. Les écrans gérance répondent 404 (pas une redirection) à un locataire, pour ne pas révéler leur existence. Le ticket se crée toujours au nom du locataire en session, même si le formulaire est falsifié.

Synchro — rejouable, idempotente, et trois découvertes qui ont dicté sa forme.

1. *Import complet puis événements.* sync:full importe 15 collections dans l'ordre des clés étrangères, lit max(change_id) **avant** la première collection et l'écrit dans sync_cursor à la fin ; sync rejoue GET /v1/sync-events?after=<curseur>&limit=500, un lot = une transaction (curseur compris). Upsert par clé primaire ERP (UUID), une ligne au source_revision plus ancien est ignorée, un delete pose deleted_at localement. Relancer n'importe quoi ne duplique rien : scripts/check-idempotent.sh enchaîne deux imports et un jeu et compare les comptes.
2. *La pagination offset de l'ERP est cassée sur deux collections.* Un parcours complet de tenant-account-entries ramène 161 603 lignes pour 133 455 identifiants distincts, et deux parcours ne donnent pas le même ensemble (ex æquo dans l'ordre serveur, aucun tri proposé). Solde obtenu pour BAIL-000170 : 3 750 CHF, contre 2 540 CHF

selon l'oracle tenant-portal-snapshots. Ces collections (écritures, loyers) sont donc importées **bail par bail** via `?lease_contract_id=`, ce qui retombe exactement sur l'oracle pour les quatre locataires de référence. Les 13 autres collections ont été vérifiées sans perte.

3. *Le flux d'événements ne contient que quatre types* (party, rental_unit, lease_contract, property) et `lease_contract` n'est **pas** le singulier de `leases` : une correspondance naïve perdrait 6 525 événements sans erreur. `entity_id` étant un UUID alors que le détail exige un `external_ref`, la résolution passe par le miroir local. Les types inconnus sont ignorés, pas fatals.

Règle du solde : toutes les écritures vivantes comptent, quel que soit leur statut — c'est la seule règle qui reproduit l'oracle sur les quatre baux. Le jeu de données ne contient que 7 soldes distincts, tous positifs : l'écran n'a donc jamais été vu avec un locataire en crédit.

4. Ce qui n'est pas fait, et pourquoi

Coupe	Pourquoi	Suite
Assistant Gemini (bonus)	« seulement si le reste tient » : le temps est allé au test d'isolation et à la synchro fiable. Le contexte par locataire existe déjà (tenant-queries), le brancher est le prochain pas.	prochaine étape n° 1 : qualifier une demande, réclamer ce qui manque, confirmer, créer le ticket
Écritures de tous les baux (défaut : baux des locataires du portail, 18 s)	l'import bail par bail des 6 525 baux prend ~26 min ; <code>--entries=all</code> existe.	lancer une fois en tâche de fond
meter-readings plafonné à 2 000 lignes sur 90 000	aucun écran ne les lit ; les écritures, elles, ne sont <i>pas</i> plafonnées car un solde tronqué serait faux.	graphique de consommation
Chemin delete testé contre un faux ERP seulement	le flux réel ne contient aucun événement delete — impossible de le démontrer en live.	—
Photos sur les tickets, notifications e-mail	l'écriture est prouvée avec du texte ; les médias ajoutent stockage et validation non testables dans le budget.	étape n° 2
Plans de paiement, relevé PDF, calendrier de maintenance	le tableau de bord répond déjà à « ce que je dois / ce qui arrive ».	étape n° 3
Mot de passe oublié, réglages, langue de/fr	comptes créés par seed : demo.	—
Synchro en tâche de fond	le bouton exécute en ligne et bloque la requête ; suffisant pour la démo.	file d'attente

Limites connues : les co-titulaires n'ont pas de ligne dans l'oracle (leur visibilité est testée, pas leur solde).

Sur la méthode. Le plan (docs/PLAN.md) a d'abord figé les contrats partagés — schéma SQLite, types ERP, trois fonctions d'interface (`getCurrentTenant`, `getCurrentUser`, `runIncrementalSync`) — puis trois agents ont travaillé en parallèle dans des dossiers disjoints (A : ERP + synchro, B : auth + tableau de bord, C : tickets + gérance), chacun dans son `worktree`, spécifié et suivi avec OpenSpec. Les frictions utiles (la pagination cassée, `lease_contract`) ont été découvertes par mesure sur l'API, pas supposées, et sont consignées dans docs/SYNC.md et CLAUDE.md.